Leksion
Abstraksioni dhe klasat abstrakte në Python

Alion Emini

kurseinformatike.com

Përmbajtja

1. Hyrje
2. Abstraksioni informal
3. Klasat abstrakte
4. Polimorfizmi dhe `collections`
5. Duck typing vs ABC
6. Testimi
7. Gabime të shpeshta

1 Hyrje

2 Abstraksioni informal

3 Klasat abstrakte

4 Polimorfizmi dhe `collections`

5 Duck typing vs ABC

6 Testimi

7 Gabime të shpeshta

8 Përmbledhje

Motivimi: sisteme të ndërlikuara, interface i thjeshtë

- Në jetën e përditshme **nuk** mendojmë për detajet e brendshme:
  - telekomanda e televizorit;
  - pedali i gazit në makinë;
  - përdorimi i bankomatit.
- Si përdorues, shohim vetëm:
  - disa **komanda** të thjeshta (*ON/OFF*, *Withdraw*, *Accelerate*...);
  - një **interface** të qartë.
- E gjithë magjia e brendshme fshihet nga ne.

Çfarë është abstraksioni?

Përkufizim

Abstraksioni është procesi me të cilin fokusohemi te **çfarë bën** një sistem dhe lëmë mënjanë shumicën e detajeve se **si** e bën atë.

- Na lejon të përdorim sisteme të ndërlikuara përmes një interface-i të thjeshtë.
- Në programim: ndërtojmë programe të mëdha duke menduar në **nivela**.
- Nuk mbytemi në detaje në çdo hap të kodit.

Abstraksioni në Python: funksione dhe metoda builtin

- Edhe në programet më të thjeshta, përfitojmë nga abstraksioni:
  - `print()` → nuk mendojmë për sistemin e I/O;
  - `len(lista)` → nuk na intereson struktura e brendshme;
  - `my_list.sort()` → nuk mendojmë për Timsort.
- Ne shohim vetëm:
  - emrin e funksionit / metodës,
  - parametra + vlerë të kthyer.
- Kjo është forca e abstraksionit: **interface i thjeshtë, zbatim i fshehur**.

Interface publik vs implementimi i brendshëm

- Për një klasë:
  - **Interface publik** → metodat që presim t'i thërrasim nga jashtë;
  - **Implementim i brendshëm** → si ruhen të dhënat, cilët algoritme përdoren, atribute private etj.
- Shembull: `BankAccount`
  - interface: `deposit()`, `withdraw()`, `get_balance()`;
  - brenda: `__balance`, log historiku, ruajtja në DB...

Abstraksioni dhe kapsulimi

Kapsulimi

Merret me **fshehjen e të dhënave** dhe mbrojtjen e gjendjes së brendshme (atribute private, `get_`/`set_`...).

Abstraksioni

Merret me **fshehjen e detajeve të logjikës**. Përdoruesit duhet të dinë *çfarë* mund të bëjnë me objektin, jo *si* realizohen hapat.

- Në praktikë: kapsulimi + abstraksioni ecin bashkë.
- Pyetja kryesore e dizajnit: *Çfarë interface-i publik ka kuptim t'u ofroj?*.

1 Hyrje

2 Abstraksioni informal
- Funksionet dhe modulet si abstraksion
- Abstraksioni përmes interface-it të klasës
- Duck typing si abstraksion informal

3 Klasat abstrakte

4 Polimorfizmi dhe `collections`

5 Duck typing vs ABC

6 Testimi

1 Hyrje

2 Abstraksioni informal
- Funksionet dhe modulet si abstraksion
- Abstraksioni përmes interface-it të klasës
- Duck typing si abstraksion informal

3 Klasat abstrakte

4 Polimorfizmi dhe `collections`

5 Duck typing vs ABC

6 Testimi

Funksionet si abstraksion

- Një **funksion** fsheh të gjithë algoritmin brenda trupit.
- Përdoruesi sheh vetëm:
  - emrin,
  - parametrat,
  - vlerën e kthyer.
- Shembuj:
  - `sum(values)`, `max(values)`;
  - `parse_date(text)`, `format_report(data)`...

Programi 1 - temperature_utils.py

```
def c_to_f(celsius):
    return celsius * 9.0 / 5.0 + 32

def f_to_c(fahrenheit):
    return (fahrenheit - 32) * 5.0 / 9.0

def _log_conversion(from_unit, to_unit, value):
    print(f"Konvertim nga {from_unit} ne {to_unit} per vleren {value}")
```

Programi 2 - weather_report.py

```
import temperature_utils as tu

def show_report(celsius):
    fahrenheit = tu.c_to_f(celsius)
    print('Temperatura aktuale:')
    print(celsius, 'C')
    print(fahrenheit, 'F')

if __name__ == '__main__':
    show_report(22)
```

- `weather_report.py` përdor vetëm `c_to_f`.
- Nuk di asgjë për formulat apo `_log_conversion`.
- Kjo është **kuti mjetesh** (modul) me interface të thjeshtë.

1 Hyrje

2 Abstraksioni informal
- Funksionet dhe modulet si abstraksion
- Abstraksioni përmes interface-it të klasës
- Duck typing si abstraksion informal

3 Klasat abstrakte

4 Polimorfizmi dhe `collections`

5 Duck typing vs ABC

6 Testimi

Programi 3 - bank_account.py

```python
class BankAccount:
    def __init__(self, owner, initial_balance=0.0):
        self.__owner = owner
        self.__balance = float(initial_balance)

    # --- Interface publik ---
    def deposit(self, amount):
        if amount <= 0: raise ValueError('Shuma duhet te jete pozitive.')
        self.__balance += amount

    def withdraw(self, amount):
        if amount <= 0: raise ValueError('Shuma duhet te jete pozitive.')
        if amount > self.__balance: raise ValueError('Fondet nuk mjaftojne.')
        self.__balance -= amount

    def get_balance(self):
        return self.__balance

    def get_owner(self):
        return self.__owner
```

Programi 4 - atm_demo.py

```python
from bank_account import BankAccount

def main():
    acc = BankAccount('Ardi', 100.0)

    print('Pronari:', acc.get_owner())
    print('Gjendja fillestare:', acc.get_balance())

    acc.deposit(50)
    print('Pas depozitimit:', acc.get_balance())

    acc.withdraw(80)
    print('Pas terheqjes:', acc.get_balance())

if __name__ == '__main__':
    main()
```

BankAccount si kontratë informale

- Interface publik:
  - `deposit(amount)`, `withdraw(amount)`;
  - `get_balance()`, `get_owner()`.
- Pjesa tjetër e programit:
  - nuk di për `__balance`, `__owner`;
  - nuk e intereson si ruhen të dhënat.
- Kemi një **abstraksion informal**:
  - Çdo `BankAccount` duhet të ofrojë kto metoda me këtë kuptim.

1 Hyrje

2 Abstraksioni informal
- Funksionet dhe modulet si abstraksion
- Abstraksioni përmes interface-it të klasës
- Duck typing si abstraksion informal

3 Klasat abstrakte

4 Polimorfizmi dhe `collections`

5 Duck typing vs ABC

6 Testimi

Programi 5 - payments.py I

```
class BankAccount:
    def __init__(self, owner, initial_balance=0):
        self.__owner = owner
        self.__balance = initial_balance

    def deposit(self, amount):
        self.__balance += amount

    def withdraw(self, amount):
        self.__balance -= amount

    def get_balance(self):
        return self.__balance
```

Programi 5 - payments.py II

```
class Wallet:
    def __init__(self):
        self.__cash = 0

    def deposit(self, amount):
        self.__cash += amount

    def get_balance(self):
        return self.__cash
def pay_salary(account, amount):
    print('Po paguajme rrogen:', amount)
    account.deposit(amount)
```

Programi 5 - payments.py III

```
def main():
    acc = BankAccount('Ardi', 100)
    w   = Wallet()

    pay_salary(acc, 50)
    pay_salary(w, 20)

    print('Gjendja ne llogari:', acc.get_balance())
    print('Gjendja ne portofol:', w.get_balance())

if __name__ == '__main__':
    main()
```

Duck typing: ideja

Shprehja klasike

If it walks like a duck and quacks like a duck, it's a duck.

- Nuk na intereson tipi formal (`BankAccount` vs `Wallet`).
- **Mjafton** që objekti ka metodën `deposit(amount)` me kuptimin e duhur.
- Kjo është **abstraksion informal** bazuar në sjellje, jo në hierarki klasash.

1 Hyrje

2 Abstraksioni informal

3 Klasat abstrakte
- Çfarë është një klasë abstrakte?
- Moduli abc: ABC dhe @abstractmethod
- Shembulli Shape: ABC + polimorfizëm

4 Polimorfizmi dhe collections

5 Duck typing vs ABC

6 Testimi

1 Hyrje

2 Abstraksioni informal

3 Klasat abstrakte

Çfarë është një klasë abstrakte?

Moduli abc: ABC dhe @abstractmethod

Shembulli Shape: ABC + polimorfizëm

4 Polimorfizmi dhe collections

5 Duck typing vs ABC

6 Testimi

Klasë abstrakte: ideja bazë

- Klasë që:
  - përmban **metoda abstrakte**;
  - **nuk** ka kuptim të instancohet direkt;
  - shërben si **model/kontratë** për nënklasat.
- Shembuj natyralë:
  - `Shape`: duhet të ketë `area()`, `perimeter()`;
  - `PaymentMethod`: duhet të ketë `pay(amount)`.

1 Hyrje

2 Abstraksioni informal

3 Klasat abstrakte
- Çfarë është një klasë abstrakte?
- Moduli abc: ABC dhe @abstractmethod
- Shembulli Shape: ABC + polimorfizëm

4 Polimorfizmi dhe collections

5 Duck typing vs ABC

6 Testimi

Programi 6 - simple_abc.py I

```python
from abc import ABC, abstractmethod

class PaymentMethod(ABC):
    @abstractmethod
    def pay(self, amount):
        pass

class CreditCardPayment(PaymentMethod):
    def pay(self, amount):
        print(f"Po paguaj {amount} me karte krediti.")

class CashPayment(PaymentMethod):
    def pay(self, amount):
        print(f"Po paguaj {amount} cash.")
```

Programi 6 - simple_abc.py II

```
def main():
    # pm = PaymentMethod()  # GABIM
    cc = CreditCardPayment()
    cs = CashPayment()

    cc.pay(50)
    cs.pay(20)

if __name__ == '__main__':
    main()
```

Roli i ABC dhe @abstractmethod

- `class PaymentMethod(ABC):`
  - i tregon Python-it që kjo është **bazë abstrakte**.
- `@abstractmethod`:
  - shënon metoda që **duhet** të implementohen në nënklasa;
  - pengon instancimin e klasës bazë nëse ka metoda abstrakte.
- Fitimi:
  - gabimet kapen më herët (në instancim, jo në mes të ekzekutimit);
  - kontrata është e dokumentuar qartë.

1 Hyrje

2 Abstraksioni informal

3 Klasat abstrakte
- Çfarë është një klasë abstrakte?
- Moduli abc: ABC dhe @abstractmethod
- Shembulli Shape: ABC + polimorfizëm

4 Polimorfizmi dhe collections

5 Duck typing vs ABC

6 Testimi

Programi 7 - shapes.py I

```
from abc import ABC, abstractmethod
import math

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

    @abstractmethod
    def perimeter(self):
        pass
```

Programi 7 - shapes.py II

```
class Circle(Shape):
    def __init__(self, radius):
        self.__radius = radius

    def area(self):
        return math.pi * self.__radius ** 2

    def perimeter(self):
        return 2 * math.pi * self.__radius
```

Programi 7 - shapes.py III

```
class Rectangle(Shape):
    def __init__(self, width, height):
        self.__width = width
        self.__height = height

    def area(self):
        return self.__width * self.__height

    def perimeter(self):
        return 2 * (self.__width + self.__height)
```

Programi 7 - shapes.py IV

```
class Triangle(Shape):
    def __init__(self, a, b, c):
        self.__a = a
        self.__b = b
        self.__c = c

    def area(self):
        s = (self.__a + self.__b + self.__c) / 2
        return math.sqrt(s * (s - self.__a) *
                         (s - self.__b) *
                         (s - self.__c))

    def perimeter(self):
        return self.__a + self.__b + self.__c
```

Programi 8 - shapes_demo.py

```
from shapes import Circle, Rectangle, Triangle

def show_shape_info(shape):
    print('Forma:', type(shape).__name__)
    print('  Siperfaqja:', shape.area())
    print('  Perimetri:', shape.perimeter())

def main():
    shapes = [ Circle(1), Rectangle(2, 5), Triangle(3, 4, 5) ]
    for s in shapes:
        show_shape_info(s)

if __name__ == '__main__':
    main()
```

Metoda jo-abstrakte në klasën abstrakte

- ABC mund të ketë edhe metoda me zbatim konkret.
- Ideale kur:
  - ka logjikë të përbashkët për të gjitha nënklasat;
  - por duam të detyrojmë implementimin e disa metodave të tjera.

Programi 9 - shapes2.py I

```
class Shape(ABC):
    @abstractmethod
    def area(self): ...

    @abstractmethod
    def perimeter(self): ...

    def describe(self):
        print(f"Kjo eshte nje {type(self).__name__} me:")
        print(f"  siperfaqe = {self.area()}")
        print(f"  perimeter = {self.perimeter()}")
```

Programi 9 - shapes2.py II

```
class Rectangle(Shape):
    def __init__(self, width, height):
        self.__width = width
        self.__height = height

    def area(self):
        return self.__width * self.__height

    def perimeter(self):
        return 2 * (self.__width + self.__height)

    def describe(self):
        print("Drejtkendeshi special:")
        super().describe()
```

1 Hyrje

2 Abstraksioni informal

3 Klasat abstrakte

4 Polimorfizmi dhe `collections`

5 Duck typing vs ABC

6 Testimi

7 Gabime të shpeshta

8 Përmbledhje

Abstraksion + polimorfizëm

- Klasa abstrakte:
  - përkufizon **kontratën** (metodat që duhen);
  - p.sh. `Shape.draw()`, `StorageBackend.save/load`.
- Nënklasat:
  - japin **zbatime të ndryshme** të kontratës.
- Funksionet:
  - punojnë me tipin abstrakt (`Shape`, `PaymentMethod`...);
  - s'u intereson lloji konkret.

Shembull: `draw(shape: Shape)`

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def draw(self):
        pass

def draw(shape: Shape):
    shape.draw()
```

- `draw` punon me `Shape`, jo me `Circle` specifik.
- Kjo e bën kodin:
  - më të qartë (kontratë e dokumentuar),
  - më të zgjerueshëm (shtojmë forma të reja pa ndryshuar funksionin).

Programi 11 - payments2.py I

```
from abc import ABC, abstractmethod

class PaymentMethod(ABC):
    @abstractmethod
    def pay(self, amount):
        pass

class CreditCardPayment(PaymentMethod):
    def __init__(self, card_number):
        self.__card_number = card_number

    def pay(self, amount):
        print(f"Po paguaj {amount} me karte ({self.__card_number}).")
```

Programi 11 - payments2.py II

```
class PayPalPayment(PaymentMethod):
    def __init__(self, email):
        self.__email = email

    def pay(self, amount):
        print(f"Po paguaj {amount} permes PayPal ({self.__email}).")

class CashPayment(PaymentMethod):
    def pay(self, amount):
        print(f"Po paguaj {amount} cash.")
```

Programi 12 - order_processing.py

```python
from payments2 import PaymentMethod, CreditCardPayment, PayPalPayment,
    CashPayment

def process_order(payment_method: PaymentMethod, total_amount):
    print("Po perpunojme porosine...")
    payment_method.pay(total_amount)
    print("Porosia u perpunua me sukses.\n")

def main():
    cc   = CreditCardPayment("1234 5678 0000 9999")
    pp   = PayPalPayment("klient@example.com")
    cash = CashPayment()

    process_order(cc,   49.90)
    process_order(pp,   12.50)
    process_order(cash, 5.00)
```

Zgjerim pa ndryshuar kodin ekzistues

- Për të shtuar `CryptoPayment`:
  - krijojmë nënklasën e re që implementon `pay()`;
  - **nuk** ndryshojmë `process_order`.
- Ky është kombinimi klasik:

abstraksion + polimorfizëm

- i cili lejon zgjerim pa prishur kodin ekzistues.

`collections.abc`: ABC-të e koleksioneve

- Moduli `collections.abc` ofron ABC për:
  - `Iterable` → çdo gjë mbi të cilën mund të bëjmë `for`;
  - `Sequence` → diçka si listë (rend, indeksim);
  - `Mapping` → diçka si dict (çelës → vlerë).
- `list`, `tuple`, `dict`, `set` etj. janë implementime konkrete të këtyre abstraksioneve.

Programi 15 - iterables_demo.py I

```python
from collections.abc import Iterable

def sum_all(values: Iterable[float]) -> float:
    total = 0.0
    for v in values:
        total += v
    return total

def main():
    nums_list  = [1, 2, 3]
    nums_tuple = (10, 20, 30)
    nums_set   = {0.5, 1.5, 2.5}

    print("Shuma (list): ",  sum_all(nums_list))
    print("Shuma (tuple):",  sum_all(nums_tuple))
    print("Shuma (set):  ",  sum_all(nums_set))
```

Programi 15 - iterables_demo.py II

```
def gen():
    for i in range(4):
        yield i * 2

print("Shuma (generator):", sum_all(gen()))
print("A eshte string Iterable?", isinstance("hello", Iterable))
```

- `sum_all` pranon **çdo** `Iterable[float]`.
- Nuk na intereson nëse është listë, tuple, set, gjenerator...

Programi 16 - mapping_demo.py

```python
from collections.abc import Mapping
from collections import defaultdict

def print_scores(scores: Mapping[str, int]) -> None:
    for name, points in scores.items():
        print(f"{name}: {points} pike")

def main():
    regular_dict = {"Ardi": 90, "Mira": 85}
    default_dict = defaultdict(int)
    default_dict["Lira"] = 100
    default_dict["Denis"] = 70

    print("Me dict normal:")
    print_scores(regular_dict)

    print("\nMe defaultdict:")
    print_scores(default_dict)
```

Programi 17 - register_demo.py I

```python
from collections.abc import Iterable

class MyRange:
    def __init__(self, start, end):
        self._start = start
        self._end = end

    def __iter__(self):
        current = self._start
        while current < self._end:
            yield current
            current += 1

Iterable.register(MyRange)
```

Programi 17 - register_demo.py II

```
def main():
    mr = MyRange(0, 3)

    print("Vlerat ne MyRange:")
    for x in mr:
        print(x)

    print("A eshte mr Iterable?", isinstance(mr, Iterable))
```

1 Hyrje

2 Abstraksioni informal

3 Klasat abstrakte

4 Polimorfizmi dhe collections

5 Duck typing vs ABC

6 Testimi

7 Gabime të shpeshta

8 Përmbledhje

Duck typing: përmbledhje

Ideja

Nuk na intereson **lloji** formal, por nëse objekti ka **metodat** dhe sjelljen që presim.

- Shembull: `pay_salary(account, amount)` kërkon vetëm një objekt me metodën `deposit()`.
- Shumë fleksibël, shumë pythonic.

Kur mjafton duck typing?

- Skripte të vogla, projekte të brendshme.
- Kur programuesit e njohin mirë njëri-tjetrin dhe mund të bien dakord verbalish për interface-in.
- Kur fleksibiliteti është më i rëndësishëm se kontrollet strikte.

Programi 18 - logger_duck.py

```python
def process_items(items, logger):
    logger.info("Po fillojme perpunimin...")
    for item in items:
        logger.debug(f"Po perpunojme: {item}")
    logger.info("Perfundoi perpunimi.")

class SimpleLogger:
    def info(self, message):
        print("[INFO]", message)

    def debug(self, message):
        print("[DEBUG]", message)

def main():
    logger = SimpleLogger()
    process_items([1, 2, 3], logger)
```

Kur na duhen ABC-të?

- Kur shkruajmë **biblioteka** / **framework**-e.
- Kur kemi një **API publike** për përdorues të tjerë.
- Kur duam që gabimet të kapen **herët** (në instancim).
- Kur kemi hierarki të qarta tipesh (`PaymentMethod`, `StorageBackend`...).

Kombinimi i të dyjave

- Python nuk të detyron të zgjedhësh:
  - mund të kesh ABC për strukturat kryesore;
  - dhe duck typing kudo tjetër.
- Qasje praktike:
  - **ABC** → për shtyllat e arkitekturës;
  - **duck typing** → për kodin e përditshëm.

1 Hyrje

2 Abstraksioni informal

3 Klasat abstrakte

4 Polimorfizmi dhe collections

5 Duck typing vs ABC

6 Testimi

7 Gabime të shpeshta

8 Përmbledhje

Abstraksioni dhe testimi

- Varësi reale (pagesa, DB, API në internet) janë:
  - të ngadalshme,
  - të brishta,
  - ndonjëherë të shtrenjta.
- Me abstraksion:
  - logjika punon me kontratën (`PaymentMethod`);
  - në test vendosim një **FakePayment**;
  - në prodhim vendosim një implementim real.

Programi 19 - payment_testing.py

```python
from abc import ABC, abstractmethod

class PaymentMethod(ABC):
    @abstractmethod
    def pay(self, amount):
        pass

class FakePayment(PaymentMethod):
    def __init__(self):
        self.called = False
        self.paid_amounts = []

    def pay(self, amount):
        print(f"[FAKE] Do te paguanim: {amount}")
        self.called = True
        self.paid_amounts.append(amount)
```

Programi 19 - payment_testing.py (vijon)

```python
def process_order(payment_method: PaymentMethod, total_amount):
    print("Po perpunojme porosine...")
    payment_method.pay(total_amount)
    print("Porosia u perpunua.\n")

def test_process_order():
    fake = FakePayment()

    process_order(fake, 49.90)

    assert fake.called, "Metoda pay() nuk u thirr!"
    assert fake.paid_amounts == [49.90], \
        "Shuma e paguar nuk eshte ajo qe prisnim!"

    print("Testi kaloi me sukses!")

if __name__ == "__main__":
    test_process_order()
```

Inversion of Control (IoC)

Ideja

Në vend që një funksion të krijojë vetë varësitë konkrete, ai pranon një **kontratë** (ABC) nga jashtë.

- Kontrolli për *cilin* implementim të përdorim i kalon thirrësit.
- Kjo njihet edhe si **dependency injection**.

Programi 20 - ioc_demo.py I

```python
from abc import ABC, abstractmethod

class PaymentMethod(ABC):
    @abstractmethod
    def pay(self, amount):
        pass

class CreditCardPayment(PaymentMethod):
    def __init__(self, card_number):
        self.__card_number = card_number

    def pay(self, amount):
        print(f"Po paguaj {amount} me karte ({self.__card_number}).")
```

Programi 20 - ioc_demo.py II

```
class FakePayment(PaymentMethod):
    def __init__(self):
        self.paid = []

    def pay(self, amount):
        print(f"[FAKE] Do te paguanim: {amount}")
        self.paid.append(amount)

def process_order(payment_method: PaymentMethod, total_amount):
    print("Po perpunojme porosine...")
    payment_method.pay(total_amount)
    print("Porosia u perpunua.\n")
```

Programi 20 - ioc_demo.py III

```python
def demo_real():
    cc = CreditCardPayment("1234 5678 0000 9999")
    process_order(cc, 29.90)

def demo_test():
    fake = FakePayment()
    process_order(fake, 29.90)
    assert fake.paid == [29.90]
    print("Testi (me FakePayment) kaloi!\n")

if __name__ == "__main__":
    demo_real()
    demo_test()
```

1 Hyrje

2 Abstraksioni informal

3 Klasat abstrakte

4 Polimorfizmi dhe collections

5 Duck typing vs ABC

6 Testimi

7 Gabime të shpeshta

8 Përmbledhje

Abstraksioni i tepërt (over-engineering)

Over-engineering

Abstraksion i tepërt ndodh kur:

- krijon shumë shtresa klasash/ABC-sh për probleme të vogla;
- abstrahon gjëra që kanë vetëm **një** implementim;
- e bën dizajnin më të ndërlikuar se ç'duhet.

Programi 21 - Abstraksion i panevojshëm

```
from abc import ABC, abstractmethod

class AbstractOperation(ABC):
    @abstractmethod
    def execute(self):
        pass

class AbstractPrintOperation(AbstractOperation):
    @abstractmethod
    def execute(self):
        pass

class SimplePrintOperation(AbstractPrintOperation):
    def __init__(self, message):
        self.__message = message

    def execute(self):
        print(self.__message)
```

Zgjidhja më e thjeshtë

```
def simple_print(message):
    print(message)
```

Rregull praktik

- **Fillo thjesht**, abstrakto vetëm kur shfaqen nevoja reale.
- Mos krijo ABC nëse s'ke të paktën **dy** implementime konkrete.

Abstraksion i paqartë (God class)

- Klasa abstrakte me shumë përgjegjësi të ndryshme.
- Përzien koncepte të paafërta.
- Është e vështirë ta përshkruash me një fjali të qartë.
- Shpesh quhet **God class**: bën gjithçka, di gjithçka.

Shembull i keq: ReportAndPrinterAndSaver

```
from abc import ABC, abstractmethod

class ReportAndPrinterAndSaver(ABC):
    @abstractmethod
    def generate_report(self, data):
        pass

    @abstractmethod
    def print_report(self, printer_name):
        pass

    @abstractmethod
    def save_to_pdf(self, filename):
        pass

    @abstractmethod
    def send_email(self, address):
        pass
```

Ndarja në abstraksione më të vogla

```
class ReportGenerator(ABC):
    @abstractmethod
    def generate(self, data):
        pass

class ReportSaver(ABC):
    @abstractmethod
    def save(self, report, filename):
        pass

class ReportSender(ABC):
    @abstractmethod
    def send(self, report, address):
        pass
```

Pyetje para se të krijosh një ABC

Check-list

- A kam të paktën **dy nënklasa** që e ndajnë këtë interface?
- A mund ta shpjegoj këtë klasë abstrakte në **një fjali të thjeshtë**?
- Interface-i është i vogël dhe i fokusuar?
- A do ta bëjë testimin më të lehtë?

Mesazh përfundimtar

- Abstraksioni është mjet për ta bërë kodin:
  - më të qartë,
  - më të lehtë për t'u mirëmbajtur.
- Një abstraksion i mirë është pothuajse i padukshëm:
  - përdoruesit nuk mendojnë për të;
  - thjesht e përdorin dhe gjërat funksionojnë natyrshëm.

1 Hyrje

2 Abstraksioni informal

3 Klasat abstrakte

4 Polimorfizmi dhe collections

5 Duck typing vs ABC

6 Testimi

7 Gabime të shpeshta

8 Përmbledhje

Çfarë mësuam në këtë leksion

- Çfarë është **abstraksioni** dhe pse na duhet.
- Abstraksioni me:
  - funksione dhe module;
  - interface-in e klasës;
  - duck typing (abstraksion informal).
- **Klasat abstrakte** (ABC) dhe `@abstractmethod`.
- Abstraksioni si **kontratë** për polimorfizmin.
- ABC-të në `collections.abc` (`Iterable`, `Mapping`...).
- Duck typing vs klasa abstrakte dhe kombinimi i tyre.
- Abstraksioni në testim: **Fake**/mock objekte dhe IoC.
- Gabime të shpeshta dhe praktika të mira në dizajnin e abstraksioneve.

Çfarë mund të provosh vetë

- Krijo një ABC `Notifier` me metodën `send(message)`.
- Implemento:
  - `EmailNotifier`, `SMSNotifier`, `PrintNotifier`.
- Shkruaj `notify_all(notifiers, message)` që përdor polimorfizëm.
- Krijo një `FakeNotifier` për testim që ruan mesazhet e dërguara.
- Eksperimento me kombinimin:
  - duck typing (pa trashëgimi);
  - vs implementim formal me ABC.

Pyetje?

Faleminderit për vëmendjen!

Pyetje, paqartësi, komente?